Statements

Java SE



Contents

Operators

Expressions

Statements

Operators

Operator Type	Operators	Description
Assignment	=, +=, -=, *=, /=	Variable များအား အစားထိုးရာတွင် အသုံးပြု
Arithmetic	+, -, *, /, %, ++, - -	Primitive Variable များအား အပေါင်၊ အနုတ်၊ အမြှောက်၊ အစား၊ အကြွင်းရှာ ရာတွင် အသုံးပြု
Relational	<, <=, >, >=, !=, ==	Primitive Variable များအား နှိုင်းယှဉ်ရာတွင် အသုံးပြု
Logical	!, &, &&, |, ||, ^	Primitive Variables များ NOT, AND, OR Logic များအား အသုံးပြုနိုင်

Precedent Of the Operators

- Expression တစ်ခုတွင် Arithmetic Operator များအား တစ်ခုထက်မက ရေးသားနိုင်ပါသည်။
- Arithmetic Operator များအား တစ်ခုထက်မက ရေးသားထားပါက Precednece မြင့်မားသော Operator အား ဦးစားပေးတွက်ချက်ပြီး ရလာသော ရလဒ်ဖြင့် ဆက်၍ တွက်ချက်ပါသည်။
- အောက်ပါ Expression အတွင်းတွင် / သည် + ထက် Precedence မြင့်မားသောကြောင့် b အား c ဖြင့်စား၍ ရလဒ်အား a ဖြင့် ပေါင်းပေးမည်ဖြစ်သည်။
- တွက်ချက်မှုအားတိကျမှုရှိစေရန် ကွင်းစကွင်းပိတ်ဖြင့် ဝန်းရံ၍ ရေးသားနိုင်ပါသည်။

```
a + b / c
a + (b / c)
```

Expression

- Java ဘာသာရပ်တွင် Variables များ၊ Operators များ အား တွဲဖက်ပြီးသော်၎င်း၊ Method များကို ခေါ်ဆိုပြီးသော်၎င်း လုပ်ဆောင်ချက်များကို ရေးသားနိုင်

- ရေးသားရာတွင်လဲ မည်သို့ရေးသားရမည်ဆိုသည်ကို တိတိကျကျ သတ်မှတ်ထားပါသည်

- ထိုသို့ရေးသားမှုများကို Expression ဟု ခေါ်ပါသည်

Expressions

```
int data = 10;
array [0] = 100;
System.out.println("Element 1 at index 0 : "
    + array[0]);

int result = 1 + 2;     // result is now 3
if(value1 == value2) {
    System.out.println("value1 == value2");
}
```

Statements

- အဓိပ္ပါယ်ပြည့်စုံသော စာကြောင်းတစ်ကြောင်းအား Statement ဟုခေါ်ဆိုသည်။
- Statement အမျိုးအစား ၃မျိုးရှိပါသည်။
- Expression Statement
  အချို့သော Expression များ၏နောက်တွင် “;” အား ရေးသားလိုက်ရုံနှင့် Statement တစ်ခုဖြစ်လာနိုင်ပြီးဂျင်းတို့အား Expression Statement ဟု ခေါ်ဆိုပါသည်။
- Declaration Statement
  အသုံးပြုလိုသည့် Variable များအား Declare လုပ်သည့် Statement များအား Declaration Statement ဟု ခေါ်ပါသည်။
- Control Flow Statement
  ပရိုဂရမ်၏ အသွားအလာများအား Control လုပ်သည့်အခါမျိုးတွင် အသုံးပြုနိုင်ပါသည်။

Expression Statements

Expression Type	Sample
Assignment Expression	value = 23445;
Increment Expression	Value ++; value --; ++ value; -- value;
Method Invocation	obj.getValue();
Object Creation	String[] array = new String[3];

Control Flows Statements

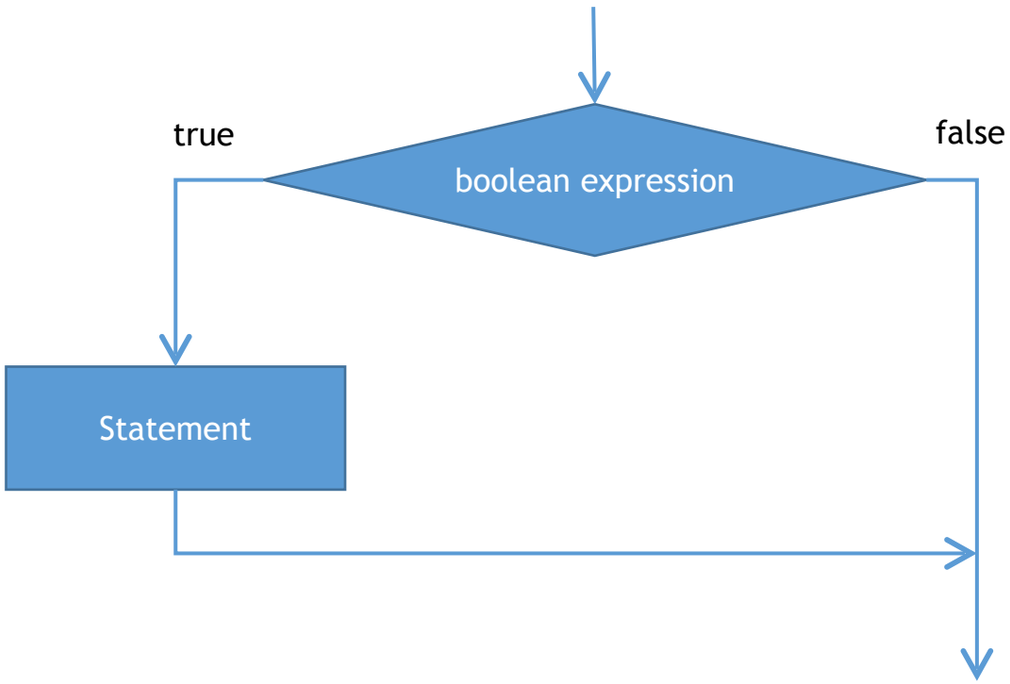
Decision Statements

- if
- if / else
- if / else if / else
- switch

Looping Statements

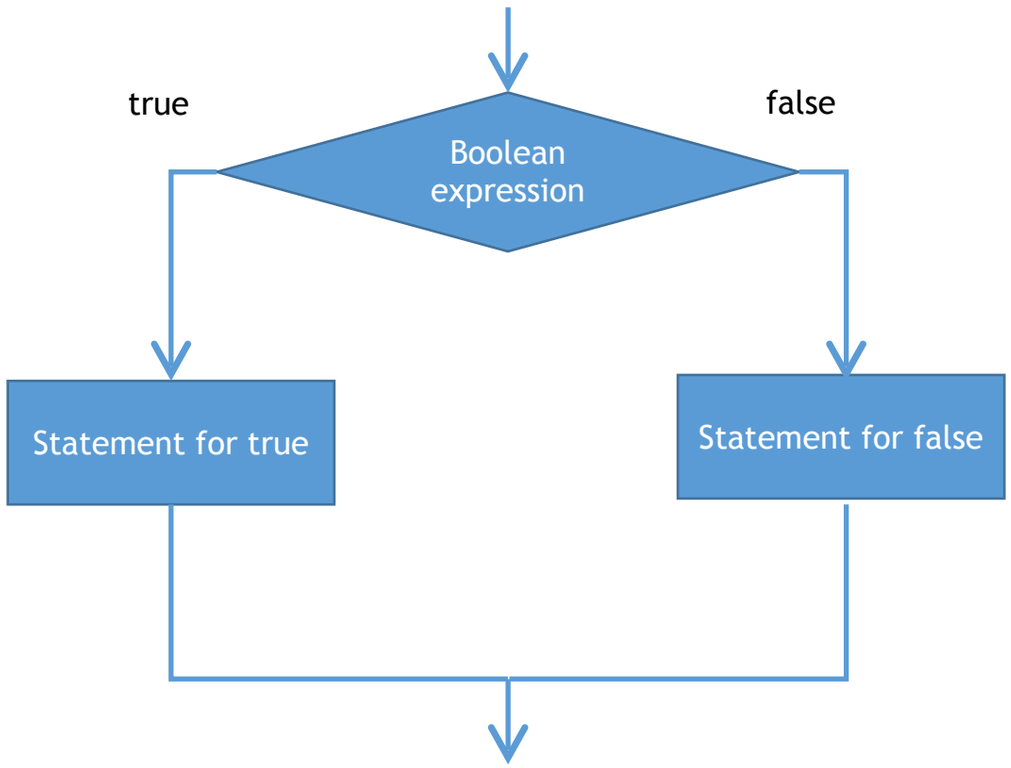
- for
- while
- do / while

if statement



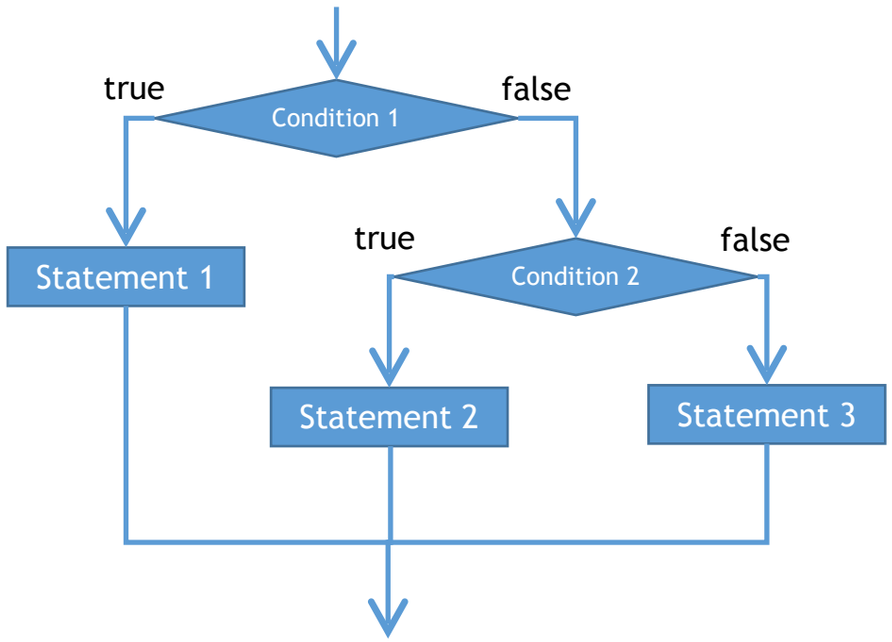

```
if (boolean expression) {
    statements ;
}
other statements;
```

if / else statement




```
if (boolean expression) {
    statements ;
} else {
    other statements;
}
```

if / else if / else




```
if (expression 1) {
    statement 1 ;
} else if (expression2) {
    statement 2;
} else {
    statement 3;
}
```

switch

```
public void showGrade(char mark) {
    switch(mark) {
    case ‘A’ :
        System.out.println(“Grade A”);
        break;
    case ‘B’ :
        System.out.println(“Grade B”);
        break;
    default :
        System.out.println(“Other Grade”);
        break;
    }
}
```

- တန်ဖိုးတစ်ခုအပေါ်လိုက်၍ ပြောင်းလည်းလုပ်ဆောင် စေလိုသည့် အခါမျိုး
- case အတွင်းတွေ့ရှိပါက break အား မတွေ့ မချင်း လုပ်ဆောင်သွားမည်
- case အတွင်းတွင် မတွေ့ရှိပါက default အား လုပ်ဆောင်မည်

switch Vs if / else if / else

- မည်သည့် အခါမျိုးတွင် switch အား အသုံးပြုသင့်၍ မည်သည့် အခါမျိုးတွင် if / else if / else အား အသုံးပြု သင့်သနည်း။
- တိကျသောတန်ဖိုးတစ်ခု အပေါ်မူတည်၍ လုပ်ဆောင်မှုအား ခွဲခြားလိုပါက ရေးသားပုံရှင်းလင်းသော switch အား ရေးသားသင့်၍၊ အတိုင်းအတာ ပမာဏ အပေါ်မူတည်၍ ခွဲခြားအသုံးပြုလိုပါက if / else if / else အား အသုံးပြုသင့်ပါသည်။

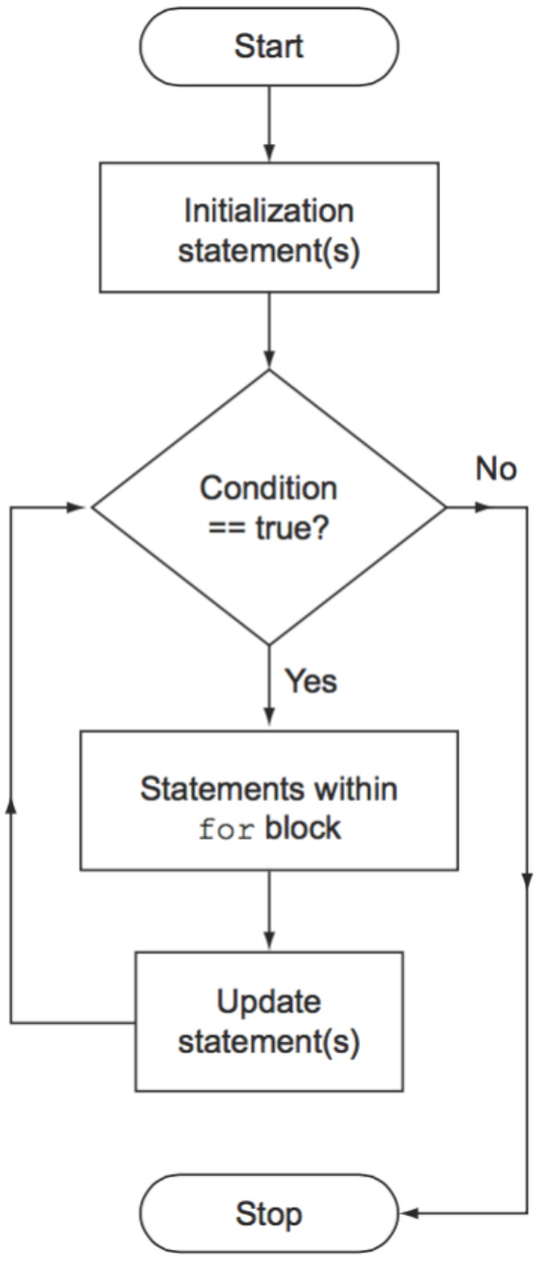
Looping Statement

- လုပ်ဆောင်ချက်များအား အကြိမ်ကြိမ်ပြုလုပ်လိုသည့် အခါအသုံးပြုရပါသည်။

- for, while နှင့် do / while တို့ရှိကြ၏။

- Array ကဲ့သို့သော Element များအား တစ်ခုစီပြုလုပ်လိုသည့် အခါမျိုးတွင် for အားဇင်း၊ boolean expression ၏ တန်ဖိုးသည် true ဖြစ်နေသ၍ အကြိမ်ကြိမ် အလုပ်လုပ်စေလိုသည့် အခါမျိုးတွင် while နှင့် do / while တို့အား အသုံးပြုနိုင်ပါသည်။

for loop



Condition
i < array.length

Initialization
int i = 0

Update
i ++

```
for (int i=0; i < array.length; i++) {
    System.out.println(array[i]);
}
```

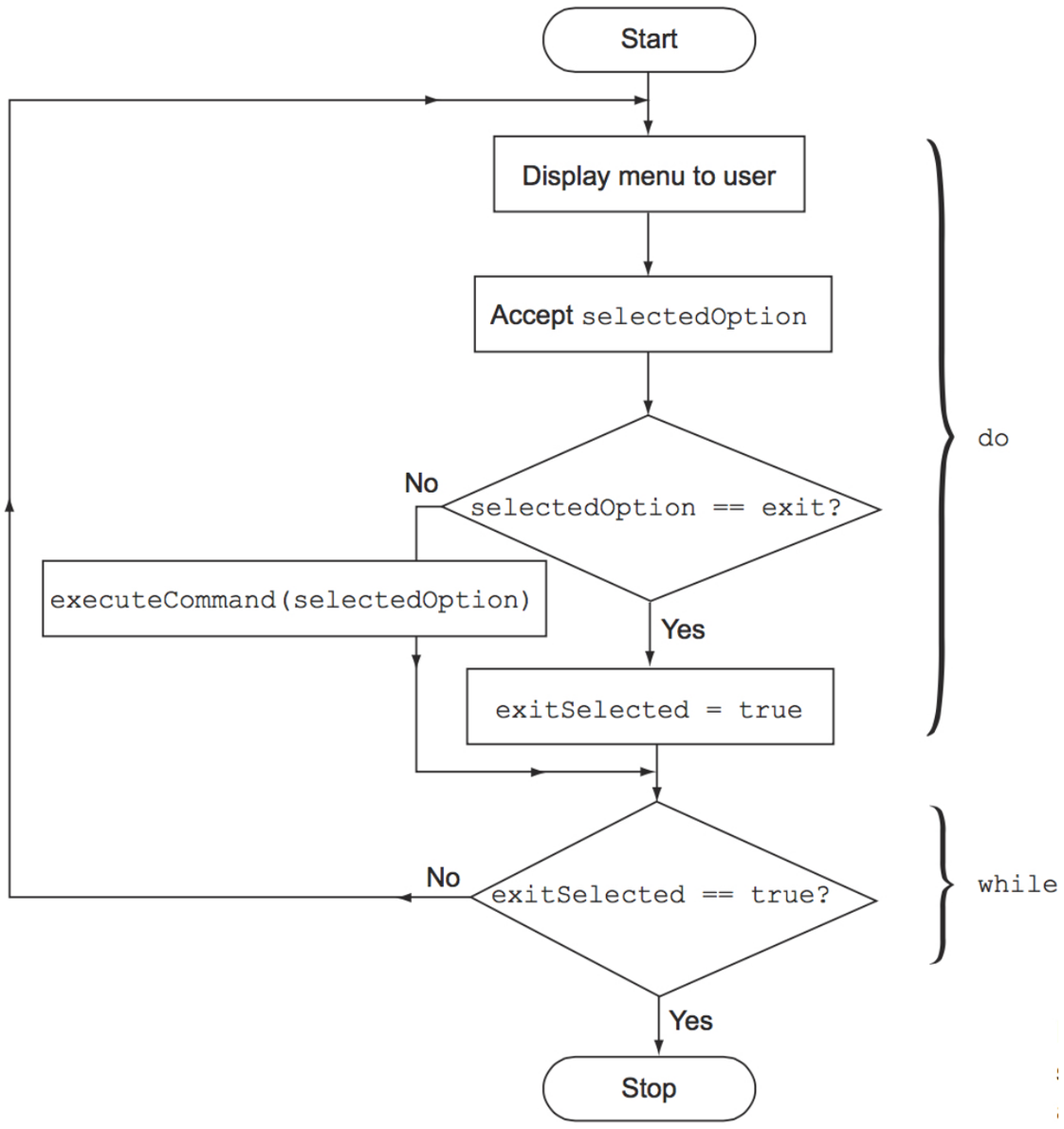
enhanced for loop

```
List<User> userList = this.getUserlist();

for (User user : userList) {
    System.out.println(user.getName());
}
```

- Java SE 5 အရောက်မှာ ဖြည့်စွက်လာတဲ့ for loop တစ်မျိုးဖြစ်ပြီး for each loop လို့လဲ ခေါ်ကြပါသည်
- Array ဒါမှမဟုတ် Collection Object တစ်ခု အတွင်းရှိ element များအား တစ်ခုခြင်း iterate လုပ်လိုသည့်အခါတွင် အသုံးပြုနိုင်သည်

do while loop



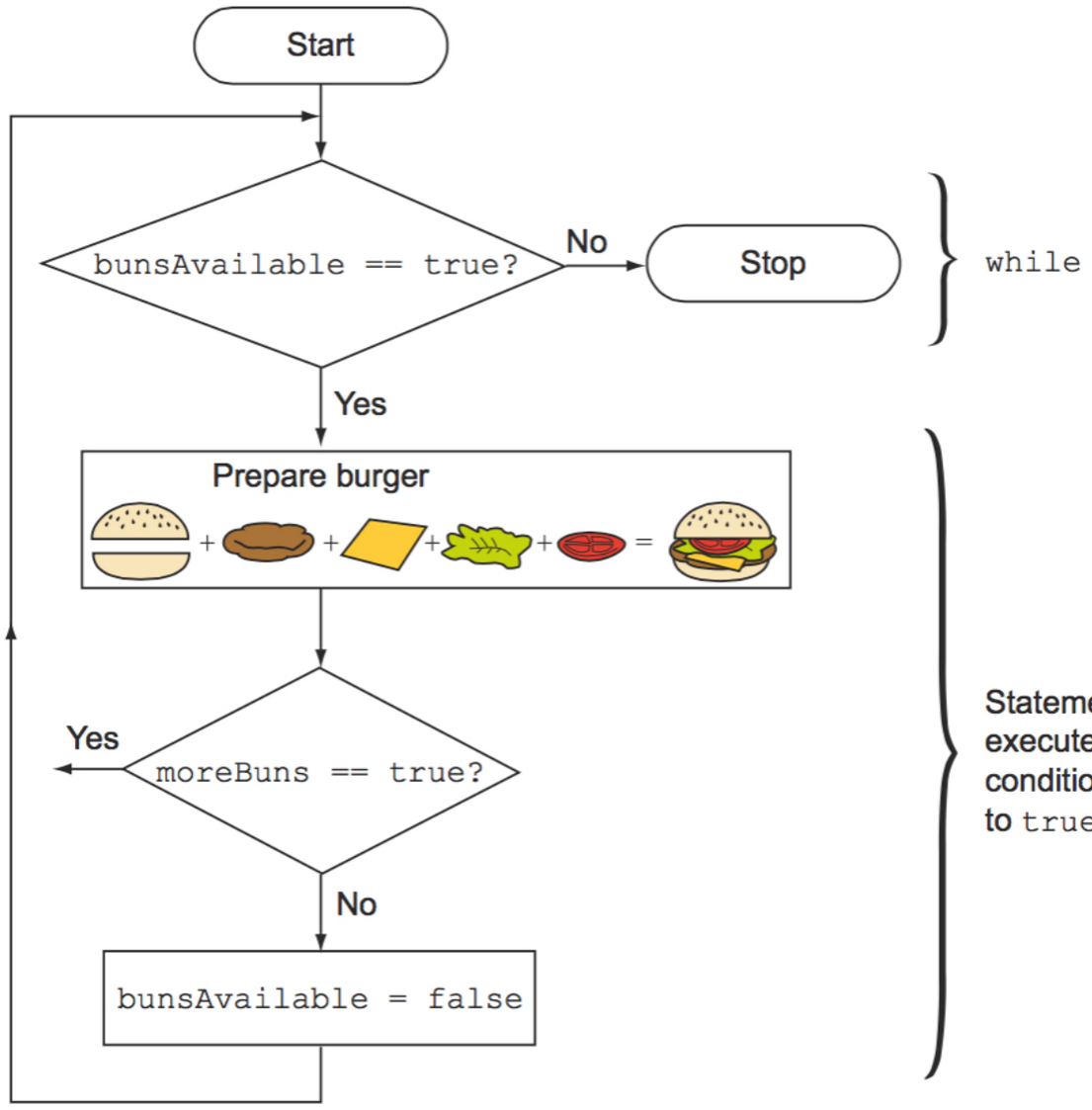

```
boolean exitSelected = false;

do {
    String selectedOption =
                displayMenuToUser();

    if(selectedOption.equals(“exit”) {
        exitSelected = true;
    }

} while ( exitSelected == false);
```

while loop




```
boolean bunsAvailable = true;

while(bunsAvailable) {

    // prepare burger

    if(nomoreBurns) {
        bunsAvailable = false;
    }
}
```

do while & while

do-while loop

```
do {
    ... code
} while (condition is true);
```

Code executes at least once, even if the `while` condition initially evaluates to `false`.

while loop

```
while (condition is true) {
    ... code
}
```

Code never executes if `while` condition initially evaluates to `false`.

branching

return

break

continue

return statement

```
public boolean isEven(int val) {
    return (val%2) == 0;
}
```

```
public void writeData(String [] array) {
    if(null == array || array.length == 0) {
        return;
    }

    for(String s : array) {
        System.out.println(s);
    }
}
```

- Return Type သည် void မဟုတ်သော method များတွင် return type ၏ တန်ဖိုးအမျိုးအစား အတိုင်း တန်ဖိုးတစ်ခုအား နောက်ဆုံးတွင် မဖြစ်မနေ return လုပ်ရပါသည်။

- Method တစ်ခုအား အဆုံးသတ်လိုပါက return statement အား အသုံးပြုနိုင်ပါသည်။

break statement

```
int i = 1;

while (i < 20) {
      System.out.println(i);

      if (i % 3 == 0) {
            break;
      }
      i++;
}
```

- Looping Statement Block အတွင်းရှိ Statement များအား ရပ်တန့်လိုပါက break statement အား အသုံးပြုနိုင်ပါသည်။

- While သို့မဟုတ် For looping block အတွင်း break statement အား တွေ့ရှိပါက လက်ရှိနေရာတွင် loop အား ရပ်တန့်ပြီး နောက်ပိုင်းရှိ statement များအား ဆက်လုပ်မည် မဟုတ်တော့ပါ။

continue statement

```
public static void main(String[] args) {
    String str ="Java is amazing programming language!";

    for (int i = 0; i < str.length(); i++) {
        if(str.charAt(i) != 'a') {
            continue;
        }
            System.out.println("Character a is at the position of " + i);
    }
}
```

- Looping statement block အတွင်း လက်ရှိလုပ်ဆောင်နေမှုများအား ရပ်၍ နောက်အကြိမ် လုပ်ဆောင်စေလိုပါက continue statement အား အသုံးပြုနိုင်ပါသည်။

- For သို့မဟုတ် While Statement block အတွင်း continue statement အား တွေ့ရှိပါက နောက်ရှိ Statement များအား ဆက်မလုပ်တော့ပဲ နောက်တစ်ကြိမ် အစမှ ပြန်၍ လုပ်မည် ဖြစ်ပါသည်။

break & Continue

`break` statement

```
for or while loop {
        :
        :
    break;
        :
        :
}
```

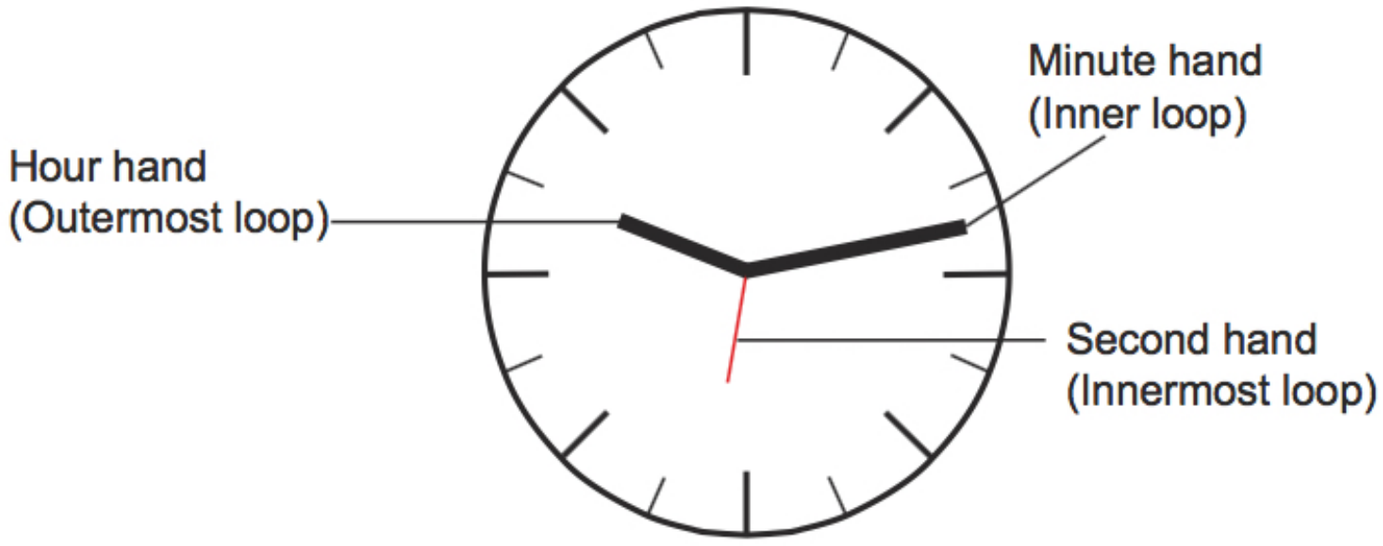
Takes control here

`continue` statement

```
for or while loop {
        :
        :
    continue;
        :
        :
}
```

Takes control to the start of next iteration

Nested Loop



1 hour = 60 minutes
1 hour = 1 complete revolution by minute hand
1 min = 60 seconds
1 min = 1 complete revolution by second hand

```
for (Hour hour : getHours() ) {
    for (Minut minut : hour.getMinuts()) {
        for(Second second : minut.getSeconds()) {
            // do something
        }
    }
}
```

Using label

- Java ဘာသာရပ်တွင် အောက်ပါ Statement များနှင့် Label များကို အသုံးပြုနိုင်ပါသည်
    - code block
    - looping statement (for, while, do while statements)
    - conditional structures (if and switch statements)
    - expressions
    - Assignments
    - return statements
    - try blocks
    - throw statements

break with label

```
public static void main(String[] args) {
    outer:
    for (int i = 0; i < 3; i++) {
        System.out.println("Outer Counter: " + i);
        for (int j = 0; j < 5; j ++) {
            if((j + 1)%3 == 0)
                break outer;
            System.out.println("Inner Counter : "  + j);
        }
    }
}
```

- အကယ်၍ looping statement အား နှစ်ထပ်အသုံးပြုထားပါက၊ သာမန် break statement အား အသုံးပြုထားပါက ထို break သည် လက်ရှိ looping statement နှင့်သာ သက်ဆိုင်မည် ဖြစ်သည်။

- သို့ရာတွင် label အား အသုံးပြုပါက break နှင့် တွဲဖက်ရေးသားထားသော label အား တပ်ဆင်ထားသည့် looping statement အထိ သက်ရောက်မှု ရှိမည် ဖြစ်ပါသည်။

continue with label

```
String str = "Java is amazing programming language!";
String [] array = str.split(" ");

search:
for (int i = 0; i < array.length; i++) {
      int j = 0;
      while (array[i].length() > j) {
            if(array[i].charAt(j ++) == 'a') {
                  continue search;
            }
      }
      System.out.println("\"" + array[i] + "\" does not contain char a.");
}
```

- Continue Statement အား Break ကဲ့သို့ပင် label အား အသုံးပြု၍ ရေးသားနိုင်ပါသည်။

- Continue နှင့် တွဲဖက်ရေးသားထားသော label အမည်နေရာရှိသော Looping Statement အထိ လက်ရှိ Statement များအား ဖြတ်ကာ နောက်တစ်ကြိမ်ပြန်လုပ်စေမည် ဖြစ်သည်။

Questions

- for နှင့် enhance for တို့၏ ကွာခြားချက်ကို ဖော်ပြပါ
- if / else if နှင့် switch တို့၏ ကွာခြားချက်ကို ဖော်ပြပါ
- while နှင့် do while တို့၏ ကွာခြားချက်ကို ဖော်ပြပါ
- for နှင့် while တို့၏ ကွာခြားချက်ကို ဖော်ပြပါ